Eötvös Loránd University

Faculty of Informatics

Dept. of Software Technology and Methodology

ELTE Informatics Chatbot Using a Small Language Model

Supervisor:
Altangerel Gereltsetseg
Assistant Professor, Ph.D

Author:
Ulziibayar Borokhul
Computer Science BSc

Budapest, 2026

Contents

Chapter 1

Introduction

1.1 Motivation

Students and staff at the Faculty of Informatics of Eötvös Loránd University face a recurring practical challenge: finding accurate, up-to-date answers to questions about the curriculum. Which courses must be completed before enrolling in a given subject? What are the rules for repeating a failed exam? How does the Neptun system handle course registration? The answers to these questions exist — scattered across faculty web pages, downloadable PDF regulations, DOCX course descriptions, and administrative guidelines — but locating the right document, navigating to the right section, and extracting a clear answer can take considerably longer than it should.

General-purpose search engines return ranked lists of pages; they do not synthesise an answer from multiple sources or understand the specific terminology of Hungarian higher education. As the volume of documentation grows each semester and more information migrates to sub-pages that are hard to discover by browsing, the gap between the information that exists and the information that students can efficiently find continues to widen.

1.2 Related Work

Several universities have built AI-powered platforms that demonstrate the viability of language models in higher education. Examining them critically, however,

reveals a consistent set of design assumptions that make them fundamentally different from the system developed in this thesis.

UM-GPT (University of Michigan) is a university-wide AI platform running on Microsoft Azure, offering access to multiple large language models to all students, faculty, and staff [1]. Its companion tool, Maizey, is built on RAG and allows any user to upload a custom dataset and create a personalised chatbot from it. While Maizey shares the same underlying retrieval mechanism as the system in this thesis, it is a general-purpose tool: users define their own knowledge base and their own purpose. The system developed here is purpose-built — its knowledge base is fixed to official faculty documents and its goal is to answer a specific and recurring student need, not to provide a blank canvas for arbitrary tasks.

ZotGPT (UC Irvine) is a broad AI services platform whose most relevant tool, ClassChat, allows instructors to configure an AI assistant around their course syllabus and materials [2]. Its primary orientation is pedagogical — supporting teaching and learning within a classroom context. It does not address the broader institutional information problem: the rules, procedures, and regulations that exist outside any single course and that students need to navigate independently.

CreateAI (Arizona State University) aggregates over 50 AI models into a single platform for building custom AI applications across the entire university [3]. Its defining characteristic is breadth, which is also its limitation for the problem at hand: a platform built to support dozens of unrelated domains simultaneously is not optimised for deep, precise retrieval over a single faculty's regulatory documents.

Across all three systems, a common profile emerges: cloud infrastructure, general-purpose scope, and significant IT investment. None are designed to answer a specific question set drawn from a single document corpus, and none were evaluated on whether they actually answer domain-specific questions correctly. This is the design space the present work inhabits.

1.3 Problem Statement

The related systems above establish that LLM-powered question answering is viable in higher education. They do not, however, provide a usable template for the ELTE Faculty of Informatics. Two concrete problems block a direct adoption.

Domain knowledge and hallucination. General-purpose LLMs have no reliable knowledge of ELTE-specific regulations, course codes, or administrative procedures. Without grounding in official faculty documents, a language model will hallucinate plausible-sounding but incorrect answers — precisely the failure mode that is most harmful in an academic advising context, where a wrong answer about prerequisites or exam rules can have real consequences for a student's academic progress.

Resource and infrastructure constraints. Deploying and maintaining a cloud-hosted platform at the scale of UM-GPT or CreateAI requires dedicated IT teams and substantial recurring costs. A single faculty cannot replicate that infrastructure. Any practical solution must run on commodity hardware — a standard laptop or a modest server — with no GPU and no cloud subscription.

No dedicated, locally deployed question-answering system currently exists for the ELTE Faculty of Informatics. Students and staff rely on direct web search, email queries to administrative offices, or informal peer advice — all of which are slow, inconsistently accurate, and do not scale as the faculty grows.

1.4 Objective

This thesis designs, implements, and evaluates a locally deployed question-answering system for the ELTE Faculty of Informatics. The central objective is to demonstrate that a system built on retrieval-augmented generation can provide students and staff with accurate, document-grounded answers to questions about the curriculum and administrative procedures — without requiring the infrastructure investment or general-purpose scope of the platforms examined in the related work. The specific goals of the system are:

- To collect and index publicly available faculty documents — including web pages, PDF regulations, and course descriptions — into a searchable local knowledge base that can be kept up to date as the faculty's web presence changes.
- To answer natural language questions about the curriculum, course requirements, and administrative procedures by retrieving relevant document pas-

sages and generating grounded responses, rather than relying on the general knowledge of a language model.

- To run entirely on commodity hardware without a GPU or cloud subscription, demonstrating that an accurate and responsive question-answering system does not require significant infrastructure investment.
- To provide a browser-based chat interface with source references, so that users can verify the origin of each answer and follow up on the source documents directly.
- To evaluate the system empirically against a no-RAG baseline, measuring whether retrieval meaningfully improves answer accuracy on questions drawn from real student queries.
- To build the system on a web service architecture that is deployment-ready, so that moving from a local prototype to a live university service would require infrastructure provisioning only, with no changes to the application code itself.

1.5 Technologies Used

Two hard constraints shaped every technology decision: the system must run on commodity hardware with 8 GB of RAM and no discrete GPU, and no query or document may leave the local machine. The following components form the complete stack.

- **Ollama**, a local model server, packages and serves large language models over a stable REST interface, handling quantisation and memory mapping automatically. It abstracts the complexity of model loading so that the rest of the application calls a single endpoint regardless of which model is active. Switching models requires only a one-line change in the configuration file, with no modifications to application code.
- **Llama 3.2 (3B) and Gemma 3 (4B)**, the two language models evaluated in this thesis, are open-weight models small enough to run on consumer hardware. Both are served at 4-bit quantisation, which reduces their memory footprint to approximately 2–2.5 GB and makes CPU-only inference feasible. This keeps

the total memory usage well within the 8 GB RAM constraint while leaving headroom for the rest of the stack.

- **all-MiniLM-L6-v2**, a compact sentence transformer, encodes both document chunks and user queries into 384-dimensional dense vectors for semantic similarity search. Because the same model encodes both the index and the query, retrieval surfaces passages that are semantically close to the question rather than merely sharing keywords. It runs efficiently on CPU and provides a strong accuracy-to-resource trade-off for English factual retrieval tasks.

- **ChromaDB**, a lightweight vector database, stores the chunk embeddings and answers nearest-neighbour queries at runtime using cosine similarity. Unlike most vector databases, it runs in-process with no separate server and persists to a plain directory on disk. This satisfies the offline-operation requirement with no additional infrastructure to install or maintain.

- **FastAPI**, an asynchronous Python web framework, exposes the RAG pipeline as a REST API and serves the browser interface as static files from the same process. It was chosen for its low boilerplate, built-in request validation, and automatic API documentation. These properties make it practical for a single developer wrapping one core pipeline into a deployable HTTP service.

- **SQLite**, a serverless embedded database engine, logs every interaction — query, answer, sources, and response time — to a single file on disk with no server process required. It integrates directly into the Python application and requires no configuration beyond a file path. For a single-machine deployment with modest log volume, this is a simpler and more portable fit than a full database engine.

- **Vanilla JavaScript** powers the browser interface as a single self-contained HTML file with embedded CSS, served directly by the FastAPI backend. Avoiding a frontend framework eliminates a build step and all Node.js dependencies entirely. This keeps deployment as simple as placing one file in the static directory, consistent with the goal of minimal operational overhead.

- **Docker**, a containerisation platform, packages the entire application — the FastAPI backend, Ollama, and all dependencies — into isolated containers

that run identically regardless of the host environment. This makes it the mechanism behind the deployability requirement: moving from a local machine to a faculty server requires only infrastructure provisioning, with no changes to the application code.

1.6 Thesis Structure

The remainder of this thesis is organised as follows.

Chapter 2 defines the system requirements. It states the functional requirements covering the capabilities exposed to the user, and the non-functional constraints including resource limits, latency, privacy, and offline operation.

Chapter 3 provides user documentation: a use-case diagram, step-by-step instructions for the web interface and command-line client, and annotated example queries.

Chapter 4 covers developer documentation. It describes the system architecture and component design, then details the full implementation across the data pipeline, embedding, RAG backend, REST API, logging, frontend, and deployment.

Chapter 5 presents the testing and evaluation of the system. It covers functional testing, the empirical RAG vs. baseline evaluation, and a user evaluation.

The Conclusion summarises the findings, reflects on the key design decisions, and outlines directions for future work.

Chapter 2

Software Requirements

This chapter defines what the ELTE IK Assistant must do and under what constraints it must operate. The functional requirements describe the capabilities the system exposes to the user. The non-functional requirements define the resource, privacy, latency, and reliability constraints that any correct implementation must satisfy. Interaction logging and the offline data pipeline are internal system concerns and do not appear as user-facing requirements.

2.1 Functional Requirements

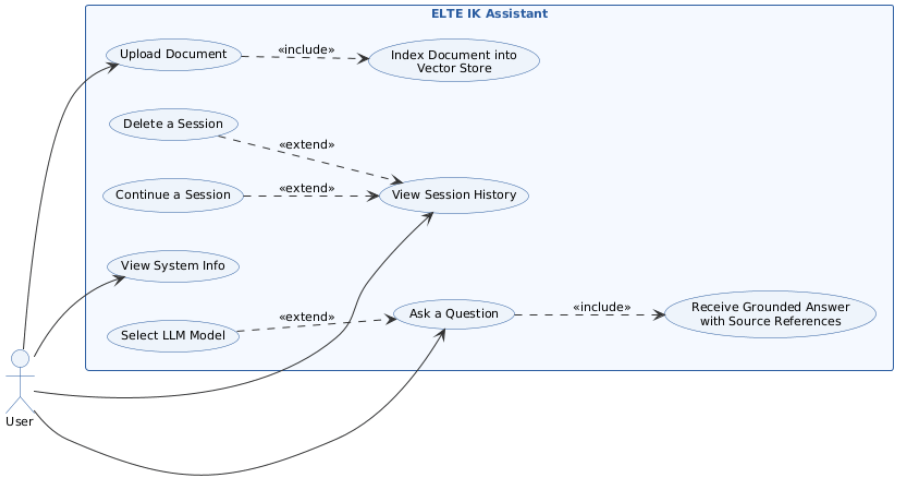

Figure 2.1: Use case diagram for the ELTE IK Assistant

Figure 2.1 illustrates the primary interactions between the user and the system. The single actor is the **User**, representing any student or staff member accessing the assistant through the browser interface. Asking a question is the central use case: it

always includes receiving a grounded answer with source references, and optionally extends to selecting a language model before submitting. Document upload always includes indexing the uploaded file into the vector store. Viewing session history optionally extends to continuing or deleting an existing session.

R-F-1 Natural language querying: the system accepts a free-text English question and returns a textual answer with source citations. When the answer is absent from the indexed documents, the system must say so explicitly rather than generating an unsupported response.

R-F-2 Model selection: the user may select which language model handles the current query. The system must apply the selected model to all subsequent queries in the session without requiring a restart.

R-F-3 Document upload: the user may upload a document that is immediately chunked, embedded, and added to the vector store, making its contents available for retrieval in the same session.

R-F-4 Embedding and retrieval: each chunk is encoded into a dense vector and stored in a local persistent vector database. At query time the top-K most similar chunks are retrieved; K is configurable.

R-F-5 Session history: the system persists conversation history across page reloads. The user may view past sessions, continue a previous session, or delete a session entirely.

R-F-6 System information: the user may view the current system status, including model availability and vector store state, without submitting a query.

R-F-7 REST API: the backend exposes a `/health` endpoint reporting system status and a `/chat` endpoint that accepts a message and returns the answer with source references. Both endpoints must be usable independently of the browser interface.

R-F-8 Web interface: a browser-based chat UI displays conversation history, source file names, model availability, and inline error messages without requiring any installation on the client side.

2.2 Non-Functional Requirements

R-NF-1 Privacy: no query, chunk, or answer may leave the local machine. The system must remain fully functional in a network-isolated environment after initial setup.

R-NF-2 Resource constraints: the system must run on 8 GB RAM with no discrete GPU, using a quantised (4-bit or 8-bit) language model and a CPU-friendly embedding model.

R-NF-3 Response latency: end-to-end response time must not exceed 90 seconds on the reference hardware. Embedding and retrieval must complete in under one second; generation typically takes 30–80 seconds on CPU.

R-NF-4 Offline operation: after the initial model pull and corpus crawl, no network connection is required. All subsequent pipeline steps run entirely from local storage.

R-NF-5 Reliability: if the language model process is unavailable, `/health` must report it and `/chat` must return an HTTP 503. The vector store and log database must not be corrupted on unexpected shutdown, and re-running the pipeline must be idempotent.

R-NF-6 Configurability: all tunable parameters — model names, K, temperature, paths, timeouts — are defined in a single configuration module. Swapping models requires only a one-line change there; changing the embedding model additionally requires rebuilding the vector store.

R-NF-7 Deployability: the system must be deployable on a faculty server without modification to the application code. All components are containerised with Docker, so that moving from a local prototype to a live university service requires infrastructure provisioning only — configuring ports, paths, and model names — with no changes to any other part of the codebase.

Chapter 3

User Documentation

This chapter describes how to set up and use the ELTE IK Assistant. Two setup paths are provided: a Docker-based path and a manual path. Choose based on your situation — Docker is better for isolated or shared deployments, the manual path is simpler for a single machine.

3.1 System Requirements

Table 3.1 lists the minimum system requirements for running the ELTE IK Assistant.

Component	**Requirement**
Operating system	Windows 10/11, macOS 12+, or Ubuntu 22.04+
RAM	8 GB minimum (16 GB recommended)
Disk space	≈6 GB (models + vector store + crawled data)
Internet access	Required during first-run setup only

Table 3.1: Minimum system requirements.

3.2 Installation

Option A — Docker

Docker Compose orchestrates all services automatically: the Ollama language model server, both model downloads, the knowledge base build, and the FastAPI application. This option is best suited for isolated or shared deployments.

First-run note: on the first `docker compose up`, the container crawls ELTE documents and builds the vector index before starting the server. This takes approximately 5–10 minutes depending on internet speed and hardware. Subsequent starts are immediate.

1. Install **Docker Desktop** (includes Docker Compose) from the official Docker website for your operating system.

2. Clone or download the project repository:

```
git clone https://github.com/borohull/Thesis.git
cd Thesis/elte_chat
```

3. Start all services:

```
docker compose up -d
```

4. Monitor first-run progress:

```
docker compose logs -f app
```

   Wait until `Application startup complete` appears in the logs.

5. Open the chat interface at `http://localhost:8000/static/index.html`.

6. To stop the containers:

```
docker compose down
```

Option B — Manual Setup

This option is best suited for running the assistant on a single local machine. Steps 1–4 are performed **once** by whoever sets up the system. End users only need step 5 (opening the browser).

Prerequisite: Python 3.10 or newer must be installed on your system.

1. Install **Ollama** from `https://ollama.com/download`, then start the service and pull both language models:

```
ollama serve
ollama pull llama3.2:3b
ollama pull gemma3:4b
```

2. Clone or download the project repository:

```
git clone https://github.com/borohull/Thesis.git
cd Thesis/elte_chat
```

3. Run the setup script. This creates a virtual environment, installs dependencies, crawls ELTE documents, and builds the vector index.

   macOS / Linux:

```
chmod +x setup.sh
./setup.sh
```

   Windows:

```
setup.bat
```

   The script prints progress at each stage. A complete first run takes approximately 5–10 minutes. Listing 3.2 shows the expected terminal output.

```
=== ELTE IK Assistant Setup ===
[1/6] Starting Ollama service...
[2/6] Pulling language models (this may take several minutes)
    ...
pulling manifest
pulling dde5aa3fc5ff: 100% 2.0 GB
...
success
pulling manifest
pulling aeda25e63ebd: 100% 3.3 GB
...
success
[3/6] Setting up Python environment...
[4/6] Crawling ELTE documents (this will take several minutes)
    ...
  [SAVE] https://inf.elte.hu/en  ->  index.html
  [SAVE] https://inf.elte.hu/en/courses  ->  courses.html
  [PDF-SAVE] https://inf.elte.hu/...  ->  regulations.pdf
  [SAVE] https://www.elte.hu/en/quaestura-office  ->  quaestura
      -office.html
  ...
Crawl complete -- saved: 482, skipped: 0, failed: 5, PDFs: 754,
    DOCXs: 46
```

```
[5/6] Building knowledge base (chunking + embedding)...
=== build_index.py: ELTE IK knowledge base builder ===
--- Phase 1: Ingestion ---
Scan: 482 files on disk, 0 in manifest
  new:       482
  updated:   0
  deleted:   0
  unchanged: 0
--- Phase 2: Embedding ---
Loaded 6128 chunks
Batches: 100% 192/192
Upserted 6128 chunks into 'elte_ik'
Collection now has 6128 documents
=== Done. Knowledge base is up to date. ===
[6/6] Setup complete.

To start the server:
  source .venv/bin/activate && uvicorn app.main:app  (macOS/
      Linux)
  .venv\Scripts\activate && uvicorn app.main:app     (Windows)

Then open: http://localhost:8000/static/index.html
```

Code 3.1: Expected terminal output during first-run setup.

On subsequent runs the crawler skips already-downloaded files and the embedding step skips unchanged chunks, so re-running setup is fast and safe.

4. Start the server:

```
# macOS / Linux
source .venv/bin/activate && uvicorn app.main:app

# Windows
.venv\Scripts\activate && uvicorn app.main:app
```

Code 3.2: Expected terminal output during first-run setup.

5. Open the chat interface at `http://localhost:8000/static/index.html`.

To verify the server is running correctly, run:

```
curl http://localhost:8000/health
```

A successful response is `{"status": "ok", "ollama": true}`. If `"ollama": false` appears, Ollama is not running or the selected model has not been pulled.

3.3 Using the Web Interface

Open `http://localhost:8000/static/index.html` in any modern browser once the server is running. The interface consists of a left sidebar listing chat sessions, a central chat panel, and a right sidebar for model selection. A status indicator in the header shows whether Ollama is reachable — if it shows as unavailable, start or restart Ollama before sending a query.

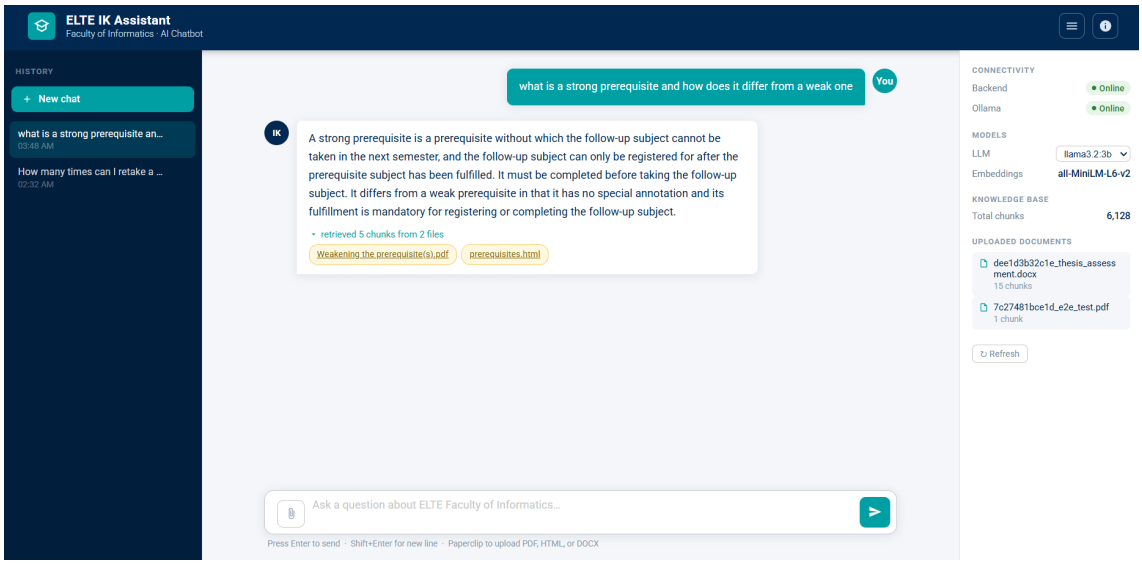

Figure 3.1: The ELTE IK Assistant web interface.

Asking a question. Type a question in the input field at the bottom of the chat panel and press **Enter** or click the send button. The assistant retrieves relevant document chunks, constructs a grounded prompt, and returns an answer along with the source file names it used. Questions should relate to the ELTE Faculty of Informatics — curriculum, regulations, procedures, and student services. On a CPU-only machine with 8 GB of RAM, typical response times are 30–80 seconds per query. The first query after startup may be slower while the model loads into memory.

Source references. Every answer displays the names of the source documents used to construct it (for example `elte_ik_regulations.html` or `TAJ.pdf`). These

are the actual files retrieved from the knowledge base. If an answer looks incorrect or incomplete, the source names indicate which documents to check directly.

Selecting a language model. The right sidebar lists all Ollama models available on the machine. Click a model name to switch for the current session. `llama3.2:3b` is the default and works on most hardware. `gemma3:4b` is also available and may produce different quality results.

Managing chat sessions. Each conversation is stored as a named session. Sessions appear in the left sidebar and persist across browser refreshes and server restarts. Click **New chat** to start a new conversation, or hover over an existing session and click the × button to delete it.

Uploading custom documents. The knowledge base can be extended with your own files. Click **Upload document** above the input field, select a PDF, HTML, or DOCX file (maximum 20 MB), and wait for the confirmation message. The document is chunked, embedded, and available for retrieval immediately — no server restart required.

3.4 Using the Command-Line Client

A command-line client is provided as an alternative to the web interface, suitable for scripting and testing. The FastAPI server must be running before starting the client:

```
1 python scripts/chat_cli.py
```

On startup the client displays a banner and connects to the server. Figure 3.2 shows the interface.

Figure 3.2: The ELTE IK Assistant command-line client.

Type a question at the `You:` prompt and press Enter. Each answer is followed by the source documents used to construct it. The client supports the following commands:

- `/commands` — display the full list of available commands.
- `/sessions` — list all past conversation sessions.
- `/resume` — select and resume a previous session.
- `/model` — switch the active language model for the current session.
- `/session` — display the current session name and active model.
- `exit` / `quit` / `Ctrl+C` — quit the client.

Each new run starts a fresh session. Conversations persist on the server and can be resumed in a later run using `/resume`.

3.5 Example Queries

The following examples illustrate the range of questions the system handles and how it responds to out-of-scope input.

Example 1 — Prerequisite query. *Question:* "What is a strong prerequisite and how does it differ from a weak one?"

Answer: A strong prerequisite is a prerequisite without which the follow-up subject cannot be taken in the next semester, and the follow-up subject can only be registered for after the prerequisite subject has been fulfilled. It must be completed before taking the follow-up subject. It differs from a weak prerequisite in that it has no special annotation and its fulfillment is mandatory for registering or completing the follow-up subject.

Sources: `Weakening the prerequisite(s).pdf`

Example 2 — Exam rules. *Question:* "How many times can I retake a failed exam in one semester?"

Answer: A student may attempt an exam up to three times per exam period. If all three attempts are failed, the course must be re-registered in a subsequent semester.

Sources: `ELTE Academic Regulations.pdf`

Example 3 — Registration procedure. *Question:* "How do I see my ranking when registering for a subject"

Answer: To see your ranking when registering for a subject, follow these steps: 1. Find the subject you want to register for. 2. Click the arrow icon at the end of the corresponding row. 3. Then click "Details" to view your score and your position in the ranking. This will show you your ranking sequence and lucky point (ranking score 1 and 2, respectively).

Sources: `registration for subjects BSc_MSc (1).pdf`

Example 4 — Out-of-scope query. *Question:* "What is the capital city of Australia?"

Answer: The information needed to answer this question is not available in the documents I have access to. I can only answer questions related to the ELTE Faculty of Informatics curriculum, regulations, and procedures.

Sources: (none)

The fourth example demonstrates the system's refusal behaviour: when no relevant chunks are retrieved, the assistant explicitly acknowledges the gap rather than generating an unsupported answer.

Chapter 4

Developer Documentation

This chapter covers the full developer documentation for the ELTE IK Assistant. It is divided into two parts: the design section describes the system architecture, key design decisions, component structure, and storage model; the implementation section details how each layer was built.

4.1 Design

This section describes the architecture of the ELTE IK Assistant, the reasoning behind its key design decisions, the module structure of the application, and the storage model used to persist data.

4.1.1 System Architecture

The ELTE IK Assistant is organised as three layers:

1. **Data pipeline** — a standalone crawler downloads raw documents from the ELTE websites; a separate ingestion and embedding pipeline cleans, chunks, and encodes those documents into vector embeddings, then stores them in a local vector database. This pipeline is run once by the system administrator and re-run whenever the source documents change.

2. **Online serving layer** — a FastAPI web server that accepts a user's natural-language question, retrieves the most relevant document chunks from the vector database, builds a prompt, calls a locally running language model, and returns the generated answer together with source references.

3. **Presentation layer** — a single-page browser UI that maintains session state, renders the conversation, and displays source file metadata alongside each answer.

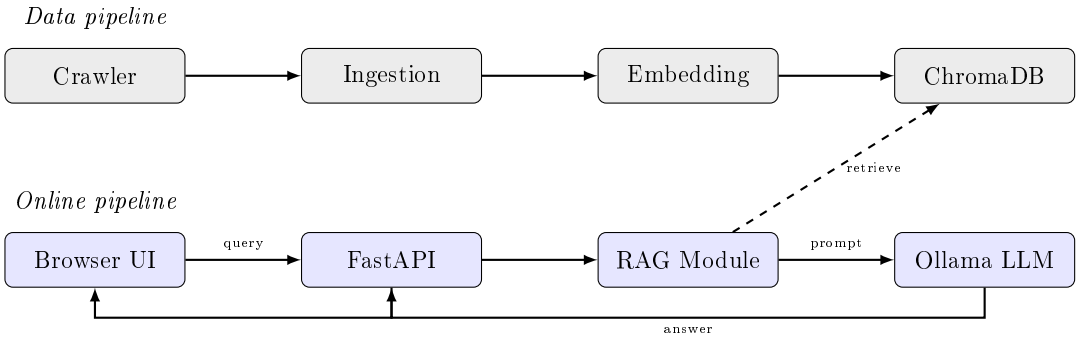

Figure 4.1: High-level architecture of the ELTE IK Assistant.

Figure 4.1 illustrates the high-level data flow between these layers. The data pipeline (top) is run once to build the vector index; the online pipeline (bottom) handles every user query at runtime.

The strict separation between the data and online pipelines is a deliberate consequence of the offline-operation requirement (R-NF-1): once the vector index is built, the serving layer requires no network access and its only disk reads are ChromaDB queries and SQLite log operations.

4.1.2 Key Design Decisions

Local LLM via Ollama. Running the language model locally is the single most consequential architectural decision. It satisfies the privacy requirement — no query or document ever leaves the machine — and the offline requirement, since no internet connection is needed at inference time. The cost is significantly higher latency compared with a cloud API: on the development hardware, median response times range from 48 to 121 seconds depending on the model. This trade-off was accepted as unavoidable given the constraints; a cloud API would have violated both requirements regardless of its performance.

Model selection. `llama3.2:3b` and `gemma3:4b` were selected as the two leading instruction-tuned open-weight models in the sub-5B parameter range available through Ollama at the time of development. The 8 GB RAM constraint rules out

7B+ parameter models at 16-bit precision (≈14 GB); both evaluated models use 4-bit quantisation, reducing their footprint to approximately 2 GB and 2.5 GB respectively, leaving sufficient headroom for the embedding model, ChromaDB index, and operating system. Both models also support instruction-following out of the box, which is necessary for the grounded-answer prompt to work reliably.

Chunking strategy. Source documents are split into 800-character chunks with a 150-character overlap. The chunk size was chosen empirically as a practical balance: large enough to contain a complete thought or regulation clause, yet small enough for the embedding model to encode with high fidelity and for three chunks to fit comfortably within the LLM context window alongside the prompt template. The 150-character overlap ensures that information near a chunk boundary appears in both adjacent chunks, so no sentence is silently dropped at a split point.

Top-k retrieval. The evaluation in Chapter 5 was conducted at $k = 5$. The production default was subsequently set to $k = 3$ to reduce inference latency on CPU-only hardware: each additional retrieved chunk lengthens the prompt and increases generation time, and the evaluation demonstrated that retrieval quality was already strong at $k = 5$, so reducing to $k = 3$ is a conservative latency trade-off with minimal impact on answer quality. At 800 characters per chunk, three chunks contribute approximately 2400 characters of context — well within the context window of both evaluated models.

Temperature. A near-zero temperature of 0.1 makes the language model near-deterministic and biased toward its highest-probability completions. For a factual question-answering assistant this is desirable: users expect consistent, precise answers rather than creative variation. The value was set to 0.1 rather than exactly 0 to avoid degenerate repetition loops that were observed during development at temperature zero, where some models produce near-infinite sequences of repeated tokens.

Containerisation with Docker. Docker Compose packages the entire stack — FastAPI, Ollama, and all dependencies — into isolated containers that run identically regardless of the host environment. This directly satisfies the deployability

requirement: moving from a local machine to a faculty server requires only infrastructure provisioning, with no changes to application code. The trade-off is added complexity for single-machine users, who must install Docker Desktop and accept a longer first-run time while models are downloaded and the knowledge base is built. This cost was considered acceptable because the manual setup path remains available as an alternative for users who prefer it.

4.1.3 Component Design

The application is composed of five Python modules in the `app/` package, each with a single responsibility, plus two data pipeline scripts that run outside the serving layer. All modules read their tunable parameters from a shared configuration module (`app/settings.py`); this dependency is not shown in Figure 4.2 to avoid cluttering the arrows, but it applies to every module listed below.

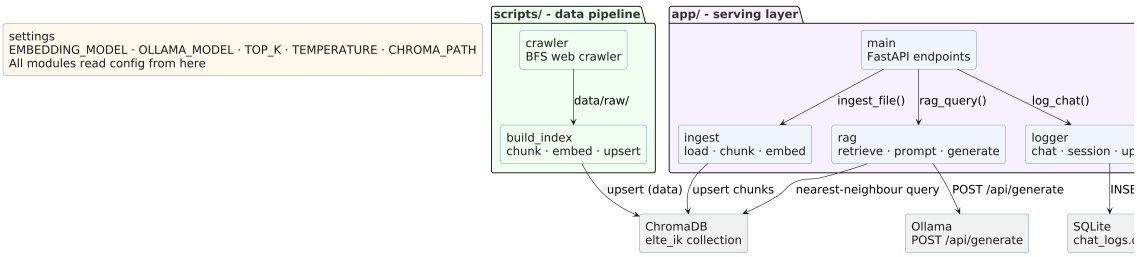

Figure 4.2: Module structure and dependencies of the ELTE IK Assistant.

Data pipeline scripts. Two scripts build the knowledge base and are never imported by the serving layer at runtime.

- **Crawler** (`scripts/crawler.py`) performs a resumable breadth-first traversal of the ELTE Faculty of Informatics website. It downloads HTML pages, PDFs, and DOCX files into `data/raw/`, computes a SHA-256 digest of each file, and records it in a manifest so that subsequent runs only re-download changed files.

- **Build index** (`scripts/build_index.py`) reads the raw files produced by the crawler, applies format-specific loaders to extract clean text, splits the text into overlapping chunks, encodes each chunk with `all-MiniLM-L6-v2`, and upserts the resulting vectors into ChromaDB. It uses the same SHA-256 manifest to process only new or changed files incrementally.

Serving layer modules. The following five modules are loaded when the FastAPI process starts and handle all runtime activity.

- **settings** (`app/settings.py`) defines every tunable constant — paths, model names, retrieval parameters, and timeouts — in one place. No business logic lives here; all other modules import from it so that a single edit propagates everywhere.
- **rag** (`app/rag.py`) is the core of the serving layer. It initialises the ChromaDB client, the ChromaDB collection, and the `all-MiniLM-L6-v2` embedding model as module-level singletons at import time, so they are loaded from disk once and reused on every request. At query time `rag_query()` encodes the user's question, retrieves the top-k most similar chunks from ChromaDB, assembles a grounded prompt, calls Ollama, and returns the answer with source references.
- **ingest** (`app/ingest.py`) handles live document ingestion for user-uploaded files. It applies the same loaders and chunking logic used by the data pipeline, then upserts the new embeddings directly into the running ChromaDB collection so that uploaded documents are available for retrieval immediately, without a server restart.
- **logger** (`app/logger.py`) owns the SQLite database. It initialises the three-table schema on first run and exposes functions for recording chat interactions, managing sessions, and logging uploaded files. It is the only module that reads from or writes to the database.
- **main** (`app/main.py`) is the FastAPI application entry point. It wires the HTTP endpoints to the other modules, mounts the static frontend files, and applies CORS middleware. It contains no business logic of its own — each endpoint delegates immediately to `rag`, `ingest`, or `logger`.

4.1.4 Storage Design

The system uses two runtime stores, accessed on every request, and two build artifacts produced by the data pipeline and consumed during index construction. All four are plain files on the local filesystem — no database server, no cloud storage, and no network filesystem are involved.

Store	Location	Contents and access pattern
ChromaDB	`data/processed/chroma_db/`	6112 chunk embeddings (384-dim float32) and metadata. Written once by the data pipeline; read on every query via approximate nearest-neighbour search.
SQLite	`data/logs/chat_logs.db`	One row per chat turn: session id, user message, assistant answer, sources, response time, and timestamp. Appended on every request; queried for session listing and history retrieval.
chunks.json	`data/processed/chunks.json`	All document chunks with text and metadata produced by the ingestion step. Written by `build_index.py` after chunking; read by the embedding step to populate ChromaDB. Not accessed at runtime.
manifest.json	`data/processed/manifest.json`	SHA-256 digests, chunk counts, and timestamps for every crawled file. Written and read by the data pipeline to detect new, updated, and deleted files during incremental re-indexing. Not accessed at runtime.

Table 4.1: Runtime stores (above double line) and data pipeline artifacts (below double line) used by the ELTE IK Assistant.

Because the entire system state is contained in these four files, the knowledge base can be backed up or migrated by copying the `data/` directory, and the serving layer can be restarted on any machine that has the directory without rebuilding the index.

4.2 Implementation

The implementation follows two phases. The data pipeline — collection, processing, and embedding — runs once offline to build the knowledge base. The online serving layer — retrieval, answer generation, and the HTTP API — runs on every user query. The subsections below follow this order, starting with project structure and configuration before covering each phase in turn.

Component	Specification
System	Acer Aspire A715-75G
CPU	Intel Core i5-10300H (8 CPUs, 2.5 GHz)
RAM	8 GB DDR4
GPU	NVIDIA GeForce GTX 1650 (Integrated)
OS	Windows 11 Enterprise 64-bit (Build 26200)
Python	3.14.3

Table 4.2: Development and test machine specifications.

4.2.1 Project Structure

The repository is organised into clearly separated concerns, as summarised in Table 4.3. The data pipeline scripts are never imported by the serving layer; the `app/` package contains everything loaded at runtime.

Path	Purpose
Setup and deployment	
`setup.sh` / `setup.bat`	First-run setup scripts for macOS/Linux and Windows. Pull Ollama models, create the virtual environment, run the crawler, and build the index.
`docker-compose.yml`	Orchestrates the Ollama service, model pull, and FastAPI application as isolated containers for server deployment.
Data pipeline (run once; not imported at runtime)	
`scripts/crawler.py`	Resumable BFS web crawler for both ELTE domains. Downloads HTML, PDF, and DOCX files into `data/raw/`.
`scripts/build_index.py`	Production pipeline script. Ingests raw files, chunks them, generates embeddings with `all-MiniLM-L6-v2`, and upserts vectors into ChromaDB incrementally using the SHA-256 manifest.
Serving layer (`app/` package, loaded at runtime)	

Path	Purpose
`app/settings.py`	Centralised configuration constants: paths, model names, retrieval parameters, and timeouts.
`app/rag.py`	Retrieval, prompt construction, and Ollama integration. Holds the ChromaDB client and embedding model as module-level singletons.
`app/ingest.py`	File loading, chunking, and live embedding for user-uploaded documents.
`app/logger.py`	SQLite-backed logging for chat interactions, sessions, and uploads.
`app/main.py`	FastAPI application entry point. Wires all HTTP endpoints; contains no business logic.
`app/static/`	Single-page HTML/CSS/JS frontend, served as a static file.
Development and evaluation (not part of the deployed system)	
`scripts/chat_cli.py`	Interactive terminal client for the chat API.
`tests/`	End-to-end pytest suite covering all API endpoints.
`notebooks/01-03`	Exploration notebooks for ingestion, embedding, and RAG pipeline development. Superseded by `build_index.py` for production use.
`notebooks/04_evaluation.ipynb`	Evaluation of retrieval and answer quality across four model/RAG configurations.
Data directories (not tracked in version control)	
`data/raw/`	Crawled source documents (HTML; PDFs in `linked_pdfs/`; DOCX in `linked_docx/`).
`data/processed/`	`chunks.json`, `manifest.json`, `pending_changes.json`, and the ChromaDB vector store.
`data/logs/`	SQLite chat log database.
`data/uploads/`	User-uploaded documents stored after ingestion.

Table 4.3: Project directory structure and file responsibilities.

4.2.2 Central Configuration

All tunable parameters are centralised in `app/settings.py` so that they can be changed in one place without touching any business logic. Every other module imports from it; no constants are defined elsewhere.

```python
from pathlib import Path

BASE_DIR             = Path(__file__).parent.parent
CHROMA_PATH          = str(BASE_DIR / "data/processed/chroma_db")
CHUNKS_PATH          = str(BASE_DIR / "data/processed/chunks.json")
MANIFEST_PATH        = str(BASE_DIR / "data/processed/manifest.json")
PENDING_CHANGES_PATH = str(BASE_DIR / "data/processed/pending_changes.json")
RAW_DIR              = str(BASE_DIR / "data/raw")
UPLOAD_DIR           = str(BASE_DIR / "data/uploads")
COLLECTION_NAME      = "elte_ik"
EMBEDDING_MODEL      = "all-MiniLM-L6-v2"
TOP_K                = 5
CHUNK_SIZE           = 800
CHUNK_OVERLAP        = 150
MAX_UPLOAD_BYTES     = 20 * 1024 * 1024   # 20 MB
ALLOWED_UPLOAD_TYPES = {
    "application/pdf", "text/html",
    "application/vnd.openxmlformats-officedocument.wordprocessingml.document",
    "application/msword", "application/octet-stream",
}
ALLOWED_EXTENSIONS   = {".pdf", ".html", ".htm", ".docx"}

OLLAMA_URL   = os.getenv("OLLAMA_URL", "http://localhost:11434/api/generate")
OLLAMA_MODEL = "llama3.2:3b"
TEMPERATURE  = 0.1
TIMEOUT_S    = 120
```

Code 4.1: Central configuration (`app/settings.py`)

Table 4.4 groups the most commonly adjusted parameters by concern.

Parameter	Default	Description
Retrieval and chunking		
TOP_K	5	Chunks retrieved per query.
CHUNK_SIZE	800	Maximum chunk size in characters.
CHUNK_OVERLAP	150	Overlap between adjacent chunks in characters.
EMBEDDING_MODEL	all-MiniLM-L6-v2	Sentence embedding model. Must be identical between build_index.py and the serving layer; a mismatch produces incorrect retrieval.
Language model		
OLLAMA_MODEL	llama3.2:3b	Default language model passed to Ollama.
TEMPERATURE	0.1	LLM sampling temperature.
TIMEOUT_S	120	Per-request timeout for Ollama calls in seconds.
Upload validation		
MAX_UPLOAD_BYTES	20 MB	Maximum accepted file size for user uploads.
ALLOWED_EXTENSIONS	.pdf .html .docx	File extensions accepted by the upload endpoint.

Table 4.4: Key configuration parameters in app/settings.py.

OLLAMA_URL is the only parameter read from the environment rather than hardcoded. Its default value points to localhost, which is correct for the manual setup path. The Docker Compose file overrides it with http://ollama:11434/api/generate so that the application container reaches the Ollama container by service name without any code change. This single environment variable is what makes the same image work in both deployment modes.

4.2.3 Data Collection and Ingestion

ELTE Faculty of Informatics publishes its information across static HTML pages, downloadable PDFs, and DOCX documents. The data pipeline collects and converts all three formats into a uniform text representation before any machine learning component is applied.

File Loaders

Each file type requires a different parsing strategy. PDFs are loaded with PyMuPDF page by page. HTML files are parsed with `BeautifulSoup`: structural noise tags (`script`, `style`, `nav`, `footer`) are stripped, then each paragraph is prefixed by its nearest heading to preserve section context. DOCX files are parsed with `python-docx`, extracting paragraphs and tables while tracking heading structure. All formats share a cleaning step that collapses blank lines and normalises whitespace.

```
def clean_text(text: str) -> str:
    text = re.sub(r"\n{3,}", "\n\n", text)
    text = re.sub(r"[ \t]+", " ", text)
    return text.strip()

def load_file(file_path: str | Path) -> List[Document]:
    suffix = Path(file_path).suffix.lower()
    if suffix == ".pdf":           return load_pdf_pymupdf(file_path
        )
    elif suffix in {".txt",".md"}: return load_txt(file_path)
    elif suffix in {".html",".htm"}: return load_html(file_path)
    elif suffix == ".docx":        return load_docx(file_path)
    else: raise ValueError(f"Unsupported file type: {suffix}")
```

Code 4.2: Text cleaning and unified file loader (`scripts/build_index.py`)

Every document is represented as a LangChain `Document` carrying its text and a metadata dictionary (file name, file type, page number, and `source_relative` path). This metadata is preserved into ChromaDB and surfaced to the user as source references on every answer.

Web Crawler

Raw documents are collected by `scripts/crawler.py`, which performs a resumable breadth-first crawl across two domains: all English pages under `inf.elte.hu` recursively, and twenty whitelisted path subtrees on `www.elte.hu` covering Neptun registration, visa procedures, student finances, and dormitory housing. Explicit path prefixes on the second domain prevent the crawler from wandering into unrelated university-wide content.

The crawler confirms each page is English via the `lang` attribute and URL path before saving it. Linked PDFs and DOCX files are downloaded separately

into `linked_pdfs/` and `linked_docx/`. A 0.5-second delay between requests limits server load. The crawler is resumable — existing files are skipped — and a `-reset` flag starts fresh. The final corpus contains 482 files.

Breadth-First Search Traversal

BFS is implemented with a `deque` as the frontier queue. It explores pages level by level, ensuring shallower and typically more important pages are crawled before deep sub-pages, while bounding memory to the width of the graph rather than its depth.

Algorithm 1 BFS Web Crawler

Require: Seeds S, allowed-domain predicate $allowed(\cdot)$
$visited \leftarrow \emptyset$
$queue \leftarrow \textsc{Deque}(S)$
while *queue* is not empty **do**
    $u \leftarrow queue.popleft()$
    if $u \in visited$ **then continue**
    end if
    $visited \mathrel{+}= \{u\}$
    if u already on disk **then** parse cached HTML; **goto** link extraction
    end if
    $html \leftarrow \textsc{Fetch}(u)$
    if *html* is English **then** save to `data/raw/`
    end if
    for each link v extracted from *html* **do**
        if v ends in `.pdf` or `.docx` **and** $allowed(v)$ **then**
            download v to `linked_pdfs/` or `linked_docx/`
        else if $allowed(v)$ **and** $v \notin visited$ **then**
            $queue.append(v)$
        end if
    end for
end while

Chunking Strategy

A language model has a limited context window, so documents are split into overlapping chunks using LangChain's `RecursiveCharacterTextSplitter`.

```
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=800,
    chunk_overlap=150,
    length_function=len,
    separators=["\n\n", "\n", ". ", " ", ""]
```

```
)
```

Code 4.3: Chunking configuration (`scripts/build_index.py`)

800 characters is large enough to contain a complete regulation clause yet small enough for the embedding model to encode with high fidelity. The 150-character overlap ensures that no sentence is silently dropped at a split boundary. The separator list defines a priority order — paragraph breaks first, then line breaks, then sentence ends — falling back to mid-word splits only as a last resort. Chunks shorter than 100 characters are discarded, and each surviving chunk is assigned a `chunk_id` of the form `<source_relative>::chunk_N`.

Incremental Ingestion with SHA-256 Manifest

Re-processing all 482 source files from scratch on every run would be wasteful. `build_index.py` therefore maintains a manifest at `data/processed/manifest.json` recording a content hash, chunk count, and timestamp for every ingested file.

The hash function used is SHA-256, which maps any file's binary content to a fixed 256-bit fingerprint. Any change to even a single byte produces a completely different fingerprint, making it a reliable change detector. SHA-256 was chosen over simpler alternatives such as file size or modification timestamp because it is robust to files that are re-downloaded with the same timestamp but different content.

On each run the script computes the hash of every file on disk and compares it against the manifest, producing three change sets: new, updated, and deleted files. Only new and updated files are re-chunked.

```
new_files     = sorted(disk_paths - manifest_paths)
deleted_files = sorted(manifest_paths - disk_paths)
updated_files = sorted(
    p for p in (disk_paths & manifest_paths)
    if manifest[p]["content_hash"] != current_files[p][1]
)
```

Code 4.4: Incremental diff logic (`scripts/build_index.py`)

The diff results are written to `pending_changes.json`. In Phase 2 of the same run, the embedding step reads this file to determine exactly which chunk IDs to delete from ChromaDB and which new vectors to insert, avoiding a full re-embed

of the collection. Once complete, `pending_changes.json` is deleted so the next run starts clean.

4.2.4 Embedding and Vector Storage

Each text chunk is converted into a numeric vector — an *embedding* — that captures its semantic meaning. Chunks with similar meaning produce similar vectors, allowing the retrieval step to find relevant content even when the user's phrasing differs from the exact wording in the source document.

Embedding Model

The chosen model is `all-MiniLM-L6-v2`, a compact sentence transformer that produces 384-dimensional vectors and runs efficiently on CPU. `build_index.py` encodes all chunks in a single batched call:

```
class EmbeddingPipeline:
    def __init__(self, model_name: str = EMBEDDING_MODEL):
        self.model = SentenceTransformer(model_name)

    def encode(self, texts: list[str]) -> np.ndarray:
        return self.model.encode(texts, show_progress_bar=True)
```

Code 4.5: Embedding pipeline (`scripts/build_index.py`)

The same model is loaded as a singleton in the serving layer so that document and query embeddings always live in the same vector space. A mismatch between the model used at index time and at query time produces silently incorrect retrieval results.

ChromaDB Vector Store

The embeddings, texts, and metadata are stored in ChromaDB, a lightweight vector database that persists to disk and requires no separate server process. A single collection named `elte_ik` is created with cosine similarity as the distance metric.

```
client = chromadb.PersistentClient(path=CHROMA_PATH)
collection = client.get_or_create_collection(
    name=COLLECTION_NAME,
    metadata={"hnsw:space": "cosine"}
```

```
)
collection.upsert(
    ids=[str(c["metadata"]["chunk_id"]) for c in chunks],
    embeddings=embeddings.tolist(),
    documents=texts,
    metadatas=[c["metadata"] for c in chunks]
)
```

Code 4.6: Collection creation and upsert (`scripts/build_index.py`)

`upsert` rather than `add` ensures that re-running `build_index.py` after new documents have been crawled updates existing chunks in place rather than creating duplicates.

Cosine Similarity

For a query embedding $\mathbf{q}$ and a chunk embedding $\mathbf{d}$, cosine similarity is defined as:

$$\text{sim}(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \, \|\mathbf{d}\|} \tag{4.1}$$

The result lies in $[-1, 1]$, where 1 means maximally similar and 0 means unrelated. It is preferred over Euclidean distance for sentence embeddings because it is invariant to vector magnitude — two chunks expressing the same concept in different amounts of text score similarly, whereas Euclidean distance would penalise the longer chunk for having a larger-norm embedding.

HNSW Index

ChromaDB indexes the stored vectors using the Hierarchical Navigable Small World (HNSW) graph [4], which performs approximate nearest-neighbour search in sub-linear time. HNSW builds a multi-layer proximity graph and navigates it greedily from a random entry point, converging on the nearest neighbours without examining every element. At 6128 stored chunks the brute-force alternative would also be fast enough, but HNSW ensures the system remains performant as the collection grows.

4.2.5 Retrieval-Augmented Generation Pipeline

The RAG pipeline is the core intelligence of the system. It consists of three steps that run on every incoming user query: retrieval, prompt construction, and LLM generation. All three are implemented in `app/rag.py`.

Retrieval

The ChromaDB client, collection, and embedding model are initialised as module-level singletons at import time, so they are loaded from disk once when the FastAPI process starts and reused across all requests. Both `SentenceTransformer` and ChromaDB's `PersistentClient` are safe for concurrent read access, so no locking is required under uvicorn's default threaded worker.

```
_client     = chromadb.PersistentClient(path=CHROMA_PATH)
_collection = _client.get_collection(name=COLLECTION_NAME)
_model      = SentenceTransformer(EMBEDDING_MODEL)

def retrieve(query: str, n_results: int = TOP_K) -> list[dict]:
    query_emb = _model.encode([query]).tolist()
    results   = _collection.query(
        query_embeddings=query_emb, n_results=n_results
    )
    return [
        {"content": doc, "metadata": meta}
        for doc, meta in zip(
            results["documents"][0], results["metadatas"][0]
        )
    ]
```

Code 4.7: Singletons and retrieval function (`app/rag.py`)

When a query arrives, the embedding model encodes it into a 384-dimensional vector and ChromaDB returns the `TOP_K` (default: 5) chunks with the highest cosine similarity to that vector.

Prompt Construction

The retrieved chunks are assembled into a structured prompt that instructs the LLM to answer strictly from the provided context and to acknowledge when the answer is absent. Each chunk is labelled with its source file name.

```python
def build_prompt(query: str, chunks: list[dict]) -> str:
    context = "\n\n".join(
        f"[Source: {c['metadata']['file_name']}]\n{c['content']}"
        for c in chunks
    )
    return (
        "You are a helpful assistant for ELTE Faculty of Informatics "
        "students. Answer the question using only the context below. "
        "If the answer is not in the context, say so.\n\n"
        f"Context:\n{context}\n\n"
        f"Question: {query}\nAnswer:"
    )
```

Code 4.8: Prompt template (`app/rag.py`)

The instruction *"If the answer is not in the context, say so"* is critical for a faculty assistant: acknowledging a gap in the indexed documents is always preferable to hallucinating a regulation or deadline.

LLM Generation via Ollama

Answers are generated by a locally running model served through Ollama's HTTP API. The default model is `llama3.2:3b`, but the system supports runtime model switching — the selected model name is passed with each request and forwarded to `call_ollama`.

```python
def call_ollama(prompt: str, model: str = OLLAMA_MODEL) -> str:
    if not check_ollama():
        raise RuntimeError("Ollama is not running.")
    payload = {
        "model": model, "prompt": prompt, "stream": False,
        "options": {"temperature": TEMPERATURE, "num_ctx": 2048},
    }
    try:
        r = requests.post(OLLAMA_URL, json=payload, timeout=
            TIMEOUT_S)
        r.raise_for_status()
        return r.json()["response"].strip()
    except requests.exceptions.Timeout:
```

```
13          raise RuntimeError(f"Ollama timed out after {TIMEOUT_S}s")
14      except requests.exceptions.HTTPError as e:
15          raise RuntimeError(
16              f"Ollama HTTP {e.response.status_code} -- "
17              f"is model '{model}' pulled?"
18          )
19      except KeyError:
20          raise RuntimeError(f"Unexpected Ollama response: {r.text
                [:200]}")
```

Code 4.9: Ollama invocation (app/rag.py)

The context window is fixed at 2048 tokens, which fits five retrieved chunks plus the prompt template with room for the generated answer. The try/except block distinguishes three failure modes — timeout (model still loading), HTTP error (model not pulled), and malformed response body — each producing a descriptive RuntimeError that the API layer converts into an HTTP 503.

End-to-End Query Function

The three steps are composed into rag_query, the single entry point called by the API layer:

```
1  def rag_query(query: str, model: str = OLLAMA_MODEL) -> dict:
2      chunks = retrieve(query)
3      prompt = build_prompt(query, chunks)
4      answer = call_ollama(prompt, model=model)
5      return {
6          "answer": answer,
7          "sources": [
8              {"chunk_id": c["metadata"]["chunk_id"],
9               "file":     c["metadata"]["file_name"]}
10             for c in chunks
11         ],
12     }
```

Code 4.10: End-to-end RAG query (app/rag.py)

The function returns both the answer and a list of source references, allowing the frontend to display which documents were used and giving the user a way to verify or follow up directly.

4.2.6 FastAPI Backend

The backend is a FastAPI application (`app/main.py`) that wires all HTTP endpoints to the `rag`, `ingest`, and `logger` modules and serves the static frontend. It contains no business logic of its own.

Method	Path	Description
GET	`/health`	Returns application status and whether Ollama is reachable.
GET	`/models`	Returns the list of models available in the local Ollama installation.
GET	`/info`	Returns the active model, embedding model, total chunk count, and list of user-uploaded documents.
POST	`/chat`	Accepts a message, optional `session_id` and `model`, runs the RAG pipeline, logs the interaction, and returns the answer with source references.
GET	`/sessions`	Returns all past sessions ordered by most recently updated.
GET	`/sessions/{id}/messages`	Returns all messages in a session in chronological order.
DELETE	`/sessions/{id}`	Deletes a session and all its associated messages.
POST	`/upload`	Validates, chunks, embeds, and indexes a user-uploaded file into the live ChromaDB collection.
GET	`/source/{filename}`	Serves the original source file from `data/raw/` or `data/uploads/` for direct inspection.

Table 4.5: API endpoints exposed by the FastAPI backend.

Chat Endpoint

The `/chat` endpoint is the primary entry point for all user queries. It delegates to `rag_query`, logs the result, and updates the session record.

```
1 @app.post("/chat", response_model=ChatResponse)
2 def chat(req: ChatRequest):
3     start = time.time()
4     try:
5         result = rag_query(req.message, model=req.model or
              OLLAMA_MODEL)
6     except RuntimeError as e:
```

```
        raise HTTPException(status_code=503, detail=str(e))

    response_ms = int((time.time() - start) * 1000)

    if req.session_id:
        log_chat(
            user_message=req.message,
            answer=result["answer"],
            sources=result["sources"],
            response_ms=response_ms,
            session_id=req.session_id,
        )
        upsert_session(req.session_id, req.message)

    return {**result, "response_ms": response_ms}
```

Code 4.11: Chat endpoint (`app/main.py`)

The model name is forwarded from the request, falling back to `OLLAMA_MODEL` if absent — this is how the frontend's model selector takes effect without requiring a server restart. If `rag_query` raises a `RuntimeError` (Ollama not running, timed out, or model not pulled), the endpoint converts it into an HTTP 503. Logging and session update only occur when a `session_id` is provided; requests from scripts or other API clients that omit it are answered without being persisted.

Upload Endpoint

The `/upload` endpoint extends the knowledge base at runtime without requiring a pipeline re-run or server restart.

```
@app.post("/upload", response_model=UploadResponse)
def upload_file(file: UploadFile = File(...)):
    ext = Path(file.filename).suffix.lower()
    if file.content_type not in ALLOWED_UPLOAD_TYPES \
            or ext not in ALLOWED_EXTENSIONS:
        raise HTTPException(status_code=415, detail="Unsupported
            file type.")

    content = file.file.read(MAX_UPLOAD_BYTES + 1)
    if len(content) > MAX_UPLOAD_BYTES:
```

```
10          raise HTTPException(status_code=413, detail="File too large
                .")
11
12      sha256 = hashlib.sha256(content).hexdigest()
13      if ingest.is_already_indexed(sha256):
14          return UploadResponse(status="already_indexed", ...)
15
16      dest = Path(UPLOAD_DIR) / f"{sha256[:12]}_{safe_name}"
17      dest.write_bytes(content)
18      report = ingest.ingest_file(dest)
19      log_upload(file_name=safe_name, sha256=sha256,
20                 size_bytes=len(content), chunk_count=report.
                       chunk_count,
21                 status="indexed")
22      return UploadResponse(status="indexed", ...)
```

Code 4.12: Upload endpoint (`app/main.py`)

The endpoint processes each upload in five stages:

1. **Type validation** — the declared content type and file extension are checked against the allowed lists in `settings.py`; only PDF, HTML, and DOCX are accepted.

2. **Size check** — the file is read into memory and rejected if it exceeds `MAX_UPLOAD_BYTES` (20 MB).

3. **Deduplication** — a SHA-256 digest is computed and checked against the manifest; if the file is already indexed the endpoint returns immediately without re-processing.

4. **Ingestion** — the file is written to `data/uploads/` and passed to `ingest.ingest_file()`, which applies the same loaders, chunking, and embedding logic as the data pipeline and upserts the vectors directly into the running ChromaDB collection.

5. **Audit log** — the upload is recorded in the `uploads` table with its digest, size, and chunk count.

The document is immediately available for retrieval with no server restart required.

CORS middleware is enabled globally so that the frontend, served as a static file by the same FastAPI process, can communicate with the API without browser origin restrictions.

4.2.7 Logging System

Every interaction is recorded in a SQLite database at `data/logs/chat_logs.db`. The schema consists of three tables.

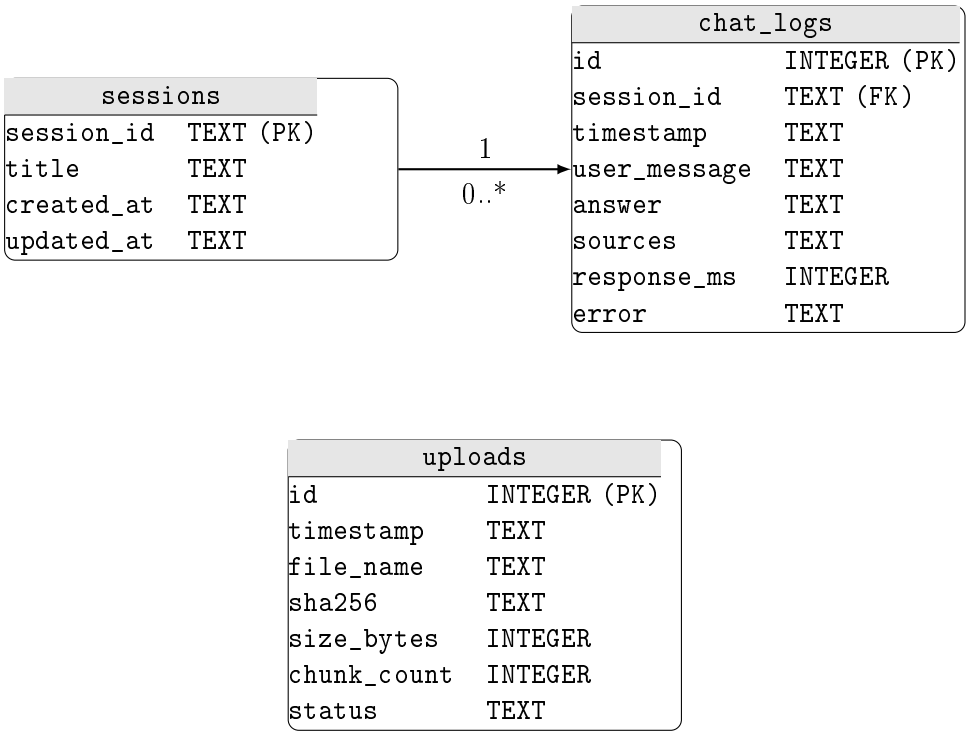

Figure 4.3: Entity-relationship diagram for the SQLite logging database.

The `chat_logs` table links each message to a `session_id`, enabling per-session history retrieval. The `sessions` table stores one row per conversation with a human-readable title derived from the opening message, so the sidebar can list sessions without scanning the full message log. The `uploads` table provides an audit trail for the live document ingestion feature, recording each file's digest and chunk count alongside its ingestion status.

4.2.8 Frontend

The user interface is a single-page HTML application served as a static file by FastAPI. Vanilla JavaScript with no framework dependency eliminates a build step and all Node.js tooling — deployment is a single file copy.

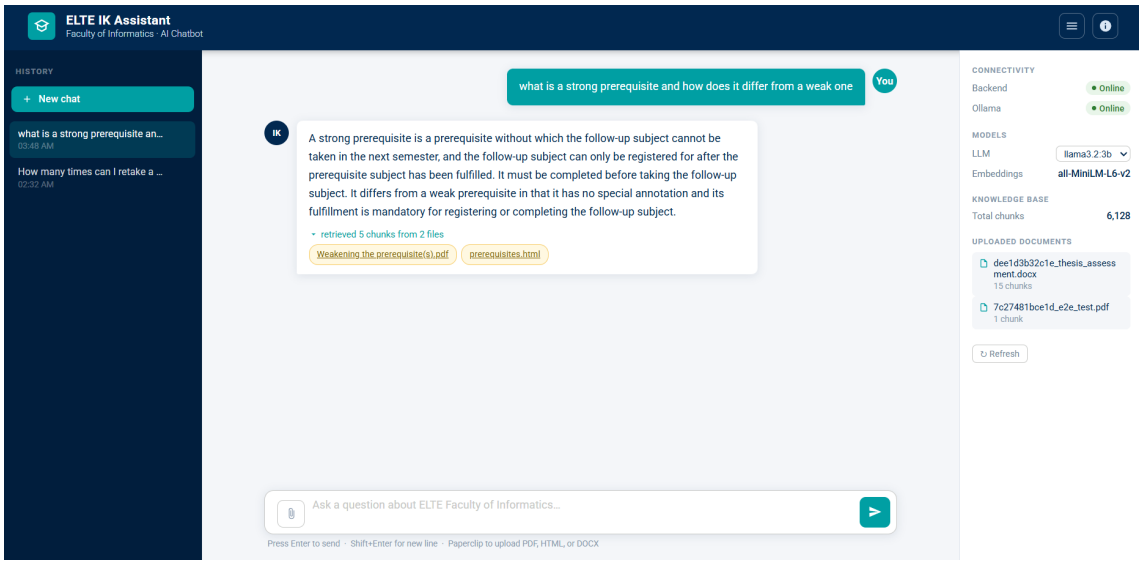

Figure 4.4: The ELTE IK Assistant web interface.

Layout and visual design

A fixed header bar, scrollable message area, and pinned input row at the bottom. The header follows the ELTE colour palette — navy blue (`#012850`) for structure, teal (`#009FA3`) for interactive elements — and shows an Ollama status indicator that polls `/health` on page load.

Session management

A UUID is generated with `crypto.randomUUID()` on page load and sent with every `/chat` request. Past sessions are listed in the left sidebar, loaded from `GET /sessions`; clicking one replays its history from `GET /sessions/{id}/messages`. `New chat` generates a fresh UUID and clears the display.

Message flow

On submit, a typing indicator is appended while `POST /chat` is in flight. On success it is replaced by the answer bubble, which includes a collapsible source list — each filename links to `GET /source/{filename}` so the user can open the original document. On HTTP 503 an inline error replaces the indicator.

Model selection

`GET /models` populates a dropdown on page load. The selected model is stored in a module-level variable and included in every `/chat` request. Switching takes effect

immediately.

Document upload

A hidden `<input type="file">` filtered to PDF, HTML, and DOCX posts to `POST /upload` as `multipart/form-data`. The response `chunk_count` is shown as confirmation, or `already_indexed` if the file is a duplicate.

Chapter 5

Testing

This chapter covers testing and evaluation of the ELTE IK Assistant. It begins with functional testing of the API endpoints and pipeline, then presents an empirical system evaluation across four model and retrieval configurations, and concludes with a user evaluation section.

5.1 Functional Testing

The project includes an end-to-end test suite located in the `tests/` directory. Tests are written using `pytest` and interact with the live system over HTTP — there are no mocks — so they exercise the full request path from the API layer through ChromaDB retrieval to Ollama inference.

Test Architecture

All tests are tagged with the `@pytest.mark.e2e` marker, which keeps them separate from any future unit tests and allows selective execution. A session-scoped `warm_up_model` fixture fires one dummy `/chat` request before the first test runs, loading the LLM into RAM so that individual tests do not time out during model initialisation. Two further session-scoped fixtures — `live_server` and `ollama_up` — check the `/health` endpoint and skip the entire session with an actionable message if the server or Ollama is not reachable.

Coverage

Table 5.1 summarises the endpoints and behaviours verified by the suite.

Endpoint	Test	Assertion	Result
`GET /health`	server up	`status == "ok"`, `ollama` key present	**Pass**
`GET /health`	Ollama online	`ollama == true`	**Pass**
`POST /chat`	basic response	200, non-empty answer, sources list, `response_ms > 0`	**Pass**
`POST /chat`	in-scope query	answer length > 20 characters	**Pass**
`POST /chat`	out-of-scope query	200, non-empty answer	**Pass**
`POST /chat`	source keys	each source contains `chunk_id` and `file`	**Pass**
`POST /chat`	session logging	session appears in `GET /sessions` after chat	**Pass**
`GET /sessions`	session created	session ID present after chat with `session_id`	**Pass**
`GET /sessions/{id}/messages`	messages retrievable	one message with correct `user_message` field	**Pass**
`DELETE /sessions/{id}`	session delete	204, session absent from list afterwards	**Pass**
`POST /upload`	valid PDF	200, `status` is `"indexed"` or `"already_indexed"`	**Pass**
`POST /upload`	duplicate file	`status == "already_indexed"` on second upload	**Pass**
`POST /upload`	invalid extension	415 Unsupported Media Type	**Pass**
`GET /info`	collection size	`total_chunks` is a non-negative integer	**Pass**
`GET /info`	embedding model	`embedding_model == "all-MiniLM-L6-v2"`	**Pass**
`GET /models`	model list	`models` is a non-empty list, `default` key present	**Pass**

Table 5.1: End-to-end test cases and their assertions.

Because LLM output is non-deterministic, no test asserts exact answer text; all assertions target response structure, status codes, and field types. All 16 test cases passed on the reference hardware configuration running `llama3.2:3b` via Ollama.

Running the Tests

Before running the suite the following prerequisites must be met: Ollama must be running (`ollama serve`), the target model must be pulled (`ollama pull llama3.2:3b`), and the FastAPI server must be started (`uvicorn app.main:app --reload`). The suite is then executed with:

```
python -m pytest tests
```

Code 5.1: Upload endpoint (`app/main.py`)

If the server or Ollama is not available, all tests are automatically skipped rather than failed, and the skip message states the exact command needed to resolve the issue.

5.2 User Evaluation

5.3 System Evaluation

The following sections present an empirical evaluation of the system across four configurations, measuring answer quality, faithfulness to retrieved source material, out-of-scope refusal behaviour, retrieval effectiveness, and response latency. The goal is to quantify how much value the RAG component adds over a plain language model baseline, and to compare the two evaluated local models under both conditions.

5.3.1 Evaluation Setup

Configurations

Four configurations were evaluated in a factorial design crossing two models with two retrieval modes:

ID	Model	RAG	Description
A1	`llama3.2:3b`	No	Baseline: model knowledge only
B1	`llama3.2:3b`	Yes	RAG with top-3 chunk retrieval
A2	`gemma3:4b`	No	Baseline: model knowledge only
B2	`gemma3:4b`	Yes	RAG with top-3 chunk retrieval

Table 5.2: The four evaluation configurations.

Both models were run locally via Ollama with 4-bit quantisation and temperature 0.1. No internet access was used during evaluation.

Test Set

The test set consists of 20 questions divided into two categories:

- **15 in-scope questions** about topics present in the crawled ELTE documents: course prerequisite rules (strong and weak), Neptun registration behaviour, faculty staff information, the Computer Science BSc programme, and international partnerships. These questions have a definite correct answer that can be verified against the source documents.
- **5 out-of-scope questions** with no connection to the faculty corpus: general knowledge (*Who won the FIFA World Cup in 2022?*), weather, cooking, geography, and humour. These questions measure whether the system correctly declines to answer rather than hallucinating.

Reference answers for in-scope questions were written manually by consulting the source documents. Each question was run through all four configurations independently; the model received no memory of previous questions.

Metrics

ROUGE-L The longest-common-subsequence F1 score between the generated answer and the reference answer [5]. Captures lexical overlap with the ground truth on a scale from 0 to 1.

Faithfulness The proportion of words in the generated answer that appear in the retrieved chunks. A high faithfulness score indicates the model is drawing on the provided context rather than confabulating.

Refusal rate For out-of-scope questions only: the fraction of queries for which the model produced a non-answer (e.g. *I don't know* or *This is outside my knowledge*). Higher is better for this category.

Retrieval hit-rate For RAG configurations: the fraction of in-scope questions for which at least one of the retrieved chunks came from the expected source document. Measures the quality of the embedding-based retrieval independently of the language model.

Response time Wall-clock latency from request to complete response, measured in milliseconds on a laptop with 8 GB RAM and no GPU.

5.3.2 Results

Overall Metric Summary

Table 5.3 reports the per-configuration averages across all 20 test questions.

ID	Model	ROUGE-L	Faithfulness	Refusal	Hit-rate	Avg time
A1	llama3.2:3b, no RAG	0.117	0.377	0.00	0.667	47.6 s
B1	llama3.2:3b, RAG	0.436	0.711	0.60	0.667	70.5 s
A2	gemma3:4b, no RAG	0.078	0.355	0.00	0.667	121.1 s
B2	gemma3:4b, RAG	0.534	0.844	0.80	0.667	61.4 s

Table 5.3: Evaluation results averaged over 20 test questions. Hit-rate is computed over in-scope questions only (15 questions); refusal rate is computed over out-of-scope questions only (5 questions).

Effect of RAG on Answer Quality

Adding RAG context produces a consistent and substantial improvement in both lexical quality and faithfulness for both models.

- **ROUGE-L** increases by a factor of 3.7× for `llama3.2:3b` (0.117 → 0.436) and by 6.9× for `gemma3:4b` (0.078 → 0.534). The larger relative gain for Gemma suggests that without retrieval it relies more heavily on parametric knowledge, which is less aligned with ELTE-specific terminology.
- **Faithfulness** roughly doubles in both cases (0.377 → 0.711 for Llama; 0.355 → 0.844 for Gemma), confirming that the retrieved chunks are genuinely shaping the generated text rather than being ignored.

The qualitative difference is visible in individual answers. For the question *"What is a strong prerequisite?"*, the no-RAG baseline (A1) produces a generic university definition with invented ELTE-specific examples that do not appear in any source document. The RAG-augmented configuration (B1) produces an answer drawn directly from the prerequisites document: *"A strong prerequisite is a prerequisite without which the follow-up subject cannot be taken in the next semester; the follow-up subject can only be registered for after the prerequisite subject has been fulfilled."* The ROUGE-L score for this question rises from 0.13 (A1) to 0.29 (B1) and faithfulness from 0.44 to 0.80.

Model Comparison

Among the RAG configurations, `gemma3:4b` (B2) outperforms `llama3.2:3b` (B1) on both answer quality metrics: ROUGE-L 0.534 vs. 0.436, faithfulness 0.844 vs. 0.711. Gemma produces more concise answers that stay closer to the retrieved context, whereas Llama tends to expand its answers with surrounding commentary.

Among the no-RAG baselines, both models score similarly poorly, with Llama slightly ahead on ROUGE-L (0.117 vs. 0.078). Neither baseline reliably answers ELTE-specific questions correctly without retrieval.

Retrieval Quality

The retrieval hit-rate is 0.667 (10 out of 15 in-scope questions) and is identical across all four configurations, since the retrieval step is determined solely by the embedding model and vector index. The five missed questions share a common cause: they ask about topics (Erasmus partnerships, CEEPUS network, Dean contact details) that are present in the HTML pages but were truncated or de-prioritised during chunking because the relevant content appeared in shallow navigation-style text rather than in a dedicated content section. This is the primary bottleneck limiting further ROUGE-L and faithfulness gains: when the correct chunk is not retrieved, the language model must fall back on parametric knowledge, which tends to be inaccurate for institution-specific facts.

Out-of-Scope Refusal

Neither baseline model refuses any out-of-scope question: both A1 and A2 answer all five irrelevant queries confidently, generating plausible but entirely off-topic responses. Among the RAG configurations, refusal rates are substantially higher based on manual review of the outputs. `llama3.2:3b` with RAG (B1) correctly refuses 3 out of 5 out-of-scope questions (60%), and `gemma3:4b` with RAG (B2) refuses 4 out of 5 (80%). In both cases the refusal is triggered by the prompt instruction to answer only from provided context: when no relevant chunks are retrieved, the model recognises the absence of grounding material and declines to answer rather than falling back on parametric knowledge.

RAG therefore has a clear secondary benefit beyond answer quality: it substantially improves the system's ability to decline out-of-scope questions. However, nei-

ther RAG configuration reaches 100% refusal, and the no-RAG baselines offer no protection at all, confirming that the prompt-level mechanism alone is not fully reliable.

Response Latency

Configuration	Avg (s)	No-RAG overhead	Req. threshold
A1 — llama3.2:3b, no RAG	47.6	—	exceeded
B1 — llama3.2:3b, RAG	70.5	+22.9 s	exceeded
A2 — gemma3:4b, no RAG	121.1	—	exceeded
B2 — gemma3:4b, RAG	61.4	−59.7 s	exceeded

Table 5.4: Average response latency. The non-functional requirement NF-4 sets a target of 30 seconds.

All four configurations exceed the 30-second latency target defined in the requirements (NF-4). This is expected on CPU-only hardware with 4-bit quantised models: the bottleneck is autoregressive token generation, which scales linearly with the number of output tokens. Two observations are worth noting. First, RAG adds latency for `llama3.2:3b` (+23 s) but *reduces* it for `gemma3:4b` (−60 s): when grounded by retrieved context, Gemma produces shorter, more focused answers, whereas without context it generates lengthy elaborations. Second, `llama3.2:3b` is substantially faster than `gemma3:4b` under both conditions, making it the more practical choice where latency matters more than marginal quality gains.

5.3.3 Discussion

RAG Effectiveness

The evaluation results confirm that RAG is effective for this use case. Grounding the language model in retrieved faculty documents more than triples ROUGE-L scores and nearly doubles faithfulness, for both of the evaluated models. The improvements are consistent and large enough to be meaningful in practice, not merely statistical noise over a 20-question benchmark.

The fundamental mechanism is straightforward: without context, neither `llama3.2:3b` nor `gemma3:4b` has reliable knowledge of ELTE-specific regulations — they were trained on general web text and have no exposure to the faculty's prerequisite rules or administrative procedures. The retrieval step injects the relevant

passage directly into the prompt, allowing the model to produce a grounded answer even though it has no parametric knowledge of the topic.

Limitations

Several limitations should be acknowledged.

Small benchmark. Twenty questions is a limited evaluation set. The in-scope questions are skewed toward prerequisite rules because that is the richest and most structured document in the corpus. Topics such as scholarship regulations, exam appeal procedures, and credit transfer rules are underrepresented.

Retrieval ceiling. The 0.667 hit-rate means one third of in-scope questions are answered without relevant context. Improving chunking strategy (e.g. heading-aware splits, larger overlap) or adding a re-ranking step could raise this ceiling and correspondingly improve ROUGE-L and faithfulness.

ROUGE-L as a proxy. ROUGE-L measures lexical overlap with a manually written reference answer. A response that is semantically correct but uses different phrasing will score poorly. For a more robust evaluation a human-judged correctness rating or a semantic similarity metric (e.g. BERTScore) would complement ROUGE-L.

Latency. All configurations exceed the 30-second target. Achieving this target on CPU-only hardware with the current models would require either a smaller model (1B parameters) or a faster inference backend. For a deployment scenario where the server runs on shared faculty hardware with a GPU, latency would drop significantly.

Recommended Configuration

Based on the evaluation results, **gemma3:4b with RAG (B2)** is the recommended production configuration: it achieves the highest answer quality (ROUGE-L 0.534, faithfulness 0.844) and, counterintuitively, lower latency than the no-RAG Gemma baseline (61 s vs. 121 s). If latency is the primary constraint, **llama3.2:3b with RAG (B1)** offers a reasonable quality-speed trade-off at 70 seconds average response time with only a modest quality reduction.

Chapter 6

Conclusion

This thesis has presented the design, implementation, and empirical evaluation of a locally deployed retrieval-augmented generation chatbot for the ELTE Faculty of Informatics. The system allows students and staff to ask natural-language questions about the curriculum, course prerequisites, exam rules, and Neptun registration procedures, and receive answers grounded in official faculty documents — without sending any data to an external server.

6.1 Summary of Contributions

The thesis has delivered the following concrete artefacts and findings.

Document collection and ingestion pipeline. A resumable breadth-first crawler collects HTML pages, PDF documents, and DOCX files from the faculty's public web presence. An incremental ingestion pipeline maintains a SHA-256 manifest so that only new or changed files need to be reprocessed on subsequent runs, reducing re-indexing cost to a fraction of a full rebuild. The resulting corpus of 482 source files is chunked into 6112 overlapping text segments and stored in a ChromaDB vector index.

Offline RAG pipeline. Each chunk is encoded into a 384-dimensional dense vector by the `all-MiniLM-L6-v2` sentence embedding model. At query time the top-3 most similar chunks are retrieved by cosine similarity and assembled into a structured prompt that instructs the language model to answer only from the

provided context. The entire pipeline runs on a standard laptop with 8 GB of RAM and no GPU.

Local language model serving. Two quantised local models — `llama3.2:3b` and `gemma3:4b` — are served by Ollama. No query, no retrieved document chunk, and no generated answer leaves the host machine at any point during inference, satisfying the privacy and offline-operation requirements that motivated the project.

Production-quality backend and frontend. A FastAPI backend exposes the RAG pipeline through a minimal REST API, logs every interaction to a local SQLite database, and serves a browser-based chat interface styled to match the faculty's visual identity. The frontend supports session history, model switching, session deletion, and per-answer source references.

Empirical evaluation. A 20-question benchmark — 15 in-scope faculty questions and 5 out-of-scope control questions — was run across all four configurations. The results establish that RAG is the decisive factor in answer quality: ROUGE-L increases by 3.7× for `llama3.2:3b` and by 6.9× for `gemma3:4b` when retrieval context is added. Source faithfulness nearly doubles in both cases. The best-performing configuration, `gemma3:4b` with RAG, achieves ROUGE-L 0.534 and faithfulness 0.844, and produces shorter, more focused answers that also reduce average response latency relative to the no-RAG Gemma baseline.

6.2 Reflection on Goals

The system meets the two primary goals stated in Chapter 1.

Privacy. All computation — embedding, retrieval, and generation — occurs on the local machine. There is no network dependency after the initial model download, and no user query reaches any external service. This satisfies the GDPR-motivated offline-operation requirement.

Domain grounding. Without RAG, neither evaluated model reliably answers ELTE-specific questions: they have no parametric knowledge of faculty prerequisite rules or Neptun registration procedures. With RAG, both models produce answers that are substantially more accurate and faithful to the source documents. The

system successfully bridges the gap between what the language models know in general and what the faculty documents say specifically.

The 30-second response latency target (NF-4) was not met: all four configurations exceed it on CPU-only hardware, with median times ranging from 47 to 121 seconds. This is a known consequence of running billion-parameter models on a laptop without GPU acceleration, and is noted as a deployment constraint rather than an architectural failure.

6.3 Limitations

Three limitations are worth restating clearly.

Retrieval ceiling. The embedding-based retrieval achieves a hit-rate of 0.667, meaning one in three in-scope questions is answered without the most relevant chunk. The missed questions concern topics whose source content appears in shallow navigation text that is not well-represented after chunking. Improving chunking strategy or adding a re-ranking step is the highest-leverage improvement available.

Out-of-scope refusal. Neither baseline model refuses any out-of-scope question; both answer all five irrelevant queries confidently without retrieval context. Among the RAG configurations, manual review of outputs shows that `llama3.2:3b` with RAG correctly refuses 3 out of 5 out-of-scope questions (60%) and `gemma3:4b` with RAG refuses 4 out of 5 (80%). The prompt-level instruction to answer only from provided context is therefore the primary driver of refusal behaviour — but it is not fully reliable. A more robust mechanism — such as a retrieval-score threshold below which the system returns a fixed "I don't know" response — is needed for a production deployment.

Benchmark scale. The 20-question test set is small and skewed toward prerequisite rules. A broader evaluation covering scholarships, exam appeals, credit transfer, and admissions would give a more complete picture of system performance.

6.4 Future Work

Several directions could extend the system beyond its current scope.

Improved retrieval. Replacing the fixed top-k retrieval with a cross-encoder reranker would improve chunk selection precision without changing the embedding index. Hybrid retrieval — combining dense vector search with a sparse BM25 index — is a well-established technique for handling exact-match queries (course codes, regulation numbers) that embedding models handle less reliably than semantic queries.

Retrieval-score threshold for refusal. A simple and robust out-of-scope detector would compare the cosine similarity of the top retrieved chunk against a calibrated threshold. If no chunk clears the threshold the system returns a fixed refusal response rather than forwarding a contentless prompt to the language model. This would eliminate the model-dependent refusal inconsistency observed in the evaluation.

Larger model on shared hardware. The latency constraint is a hardware constraint, not a software one. If the system were deployed on a faculty server with a mid-range GPU (e.g. NVIDIA RTX 3060, 12 GB VRAM), a 7B-parameter model could run at sub-10-second latency, bringing the system well within the original 30-second target while further improving answer quality.

Multilingual support. The faculty serves students who write questions in Hungarian as well as English. Replacing `all-MiniLM-L6-v2` with a multilingual sentence encoder (e.g. `paraphrase-multilingual-MiniLM-L12-v2`) and a multilingual instruction-tuned model would extend the system to Hungarian-language queries without re-indexing the corpus.

User feedback loop. The SQLite interaction log provides a natural foundation for a feedback mechanism: users could mark an answer as helpful or unhelpful. Aggregated feedback could be used to identify retrieval failures, surface low-quality chunks for manual review, and prioritise document updates in the corpus.

Bibliography

[1] University of Michigan. *Generative AI at U-M*. https://genai.umich.edu. Accessed: 2026-04-30. 2025.

[2] University of California, Irvine. *ZotGPT: UCI AI Services Platform*. https://zotgpt.uci.edu. Accessed: 2026-04-30. 2025.

[3] Arizona State University. *CreateAI Platform*. https://ai.asu.edu/create-ai-platform-product-release-notes. Accessed: 2026-04-30. 2025.

[4] Yury A. Malkov and Dmitry A. Yashunin. "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.4 (2020), pp. 824–836. DOI: 10.1109/TPAMI.2018.2889473.

[5] Chin-Yew Lin. "ROUGE: A Package for Automatic Evaluation of Summaries". In: *Text Summarization Branches Out*. Association for Computational Linguistics, 2004, pp. 74–81.

List of Figures

List of Tables

List of Algorithms

List of Codes